Synthetic Is All You Need?

Jonathan Zmora* Lior Vinman*

The Stein Faculty of Computer and Information Science
Ben-Gurion University of the Negev, Israel
{zmoraj, vinmanl}@post.bgu.ac.il

Abstract

Transferring computer vision algorithms from simulated environments to real-world applications remains a significant challenge due to large domain gaps caused by lighting, textures, sensor noise, and other physical inconsistencies. In this work, we tackle the synthetic-to-real generalization problem of chessboard state classification. First, we address the visual domain gap between synthetic and real chessboard images by engineering a highly realistic synthetic dataset via a 3D rendering pipeline using Blender. Training on this high-fidelity dataset yields significant zero-shot performance gains over a naively rendered synthetic dataset when evaluated with a ResNet18 [3] baseline model. For our primary zero-shot evaluation, we propose an architecture utilizing a local per-square ConvNeXt [4] feature extractor coupled with a Transformer Encoder [7] to capture global board context. To adapt this knowledge to real-world images, we transition to a transfer-learning approach, fine-tuning the ConvNeXt backbone's final stage with a classification head on a limited set of real-world images. Across both phases, we integrate an Integer Linear Programming (ILP) solver as a fundamental architectural component to enforce strict chessboard state constraints. This solver successfully rectifies illegal predictions, driving our final model to a 97.3% accuracy on the real-world test dataset. Our code is available at https://github.com/JonathanZmora/ChessClassification

*Equal contribution.

1 Introduction

Given a single RGB image of a physical chessboard, the primary objective of this task is to classify each of the 64 individual board squares into one of the possible piece classes or identify it as an empty square. The system must then utilize these per-square predictions to output the corresponding board state, commonly represented in Forsyth-Edwards Notation (FEN) [8], and construct the complete chessboard diagram.

Digitizing over-the-board chess games is crucial for game analysis, broadcasting, and archival purposes. However, training robust deep neural networks to perform this task requires massive amounts of labeled data. Manually annotating the exact class and location of up to 32 pieces across thousands of chessboard images is prohibitively expensive and prone to human error. Therefore, utilizing procedurally generated synthetic data is a highly attractive alternative.

However, models trained exclusively on naively generated synthetic data typically suffer from a steep performance drop when applied to real-world images, due to the *sim-to-real domain gap*. This domain gap is based on physical phenomena absent in basic 3D synthetic renderings: variable room lighting, specular highlights, realistic textures, sensor noise, poor image quality, and other physical characteristics.

Another fundamental challenge lies in the formulation of the problem. Treating the problem as a single whole-board classification task involves a very large output space, rendering direct network optimization impractical. In contrast, decomposing the problem into 64 isolated per-square classification tasks drastically simplifies the optimization process to a standard 13-class problem. However, this isolated approach discards critical global board context, ignoring the inherent structural rules of chess and the spatial relationships between pieces that could otherwise aid the network in resolving ambiguous visual features.

Our goal is to successfully bridge the sim-to-real domain gap by curating a highly realistic synthetic training dataset, while developing a neural network architecture capable of leveraging both localized per-square features and the global spatial context of the board.

The main contributions of this work are as follows:

- **Synthetic Data Rendering Pipeline:** We engineered a highly realistic synthetic chessboard dataset using an extensible and adaptable 3D Blender rendering pipeline that accurately models physical phenomena inherent to real-world environments. We make this generation pipeline publicly available to facilitate future research, and demonstrate that this data-centric approach significantly improves zero-shot transfer performance compared to low-fidelity synthetic datasets.

- **Hybrid Architecture:** We propose a hybrid neural network architecture that uses a ConvNeXt local feature extractor coupled with a Transformer Encoder, empirically demonstrating that the integration of global spatial board context improves zero-shot chessboard state classification. Furthermore, we show that this robust synthetic foundation enables highly efficient domain adaptation; fine-tuning only the final stage of the ConvNeXt backbone and a linear classification head on a limited set of real-world images is sufficient to achieve exceptional real-world performance.
- **Logically Constrained Inference:** We formulate and integrate an Integer Linear Programming (ILP) solver as a final logical constraint layer. By mathematically enforcing strict chessboard state constraints, the solver successfully rectifies illegal board state predictions, driving our final model to a test accuracy of 97.3%.

2 Related Work

2.1 Chessboard State Classification

The task of digitizing chessboard states from images has been explored through various traditional and deep learning paradigms. Wölflein and Arandjelović [9] achieved high classification accuracy on purely synthetic datasets. Although demonstrating great results, synthetic chessboard state classification is a fundamentally easier task than real chessboard state classification. Masouris et al. [5] attempted to train end-to-end models for real chessboard state classification directly on real-world chessboard images. However, this approach is limited by the immense volume of labeled data needed for a robust solution. Our work directly addresses this annotation bottleneck. We approach the problem through a sim-to-real lens, focusing on generating highly robust synthetic data that transfers to the real world without requiring massive real chessboard image datasets.

2.2 The Sim-To-Real Gap

Generating synthetic data to train models for physical-world deployment is a cornerstone of modern computer vision. The foundational work on Domain Randomization by Tobin et al. [6] demonstrated that neural networks can generalize to real-world environments by training on simulated images with heavily randomized textures and lighting. While their approach proves that the "reality gap" can be bridged using simulated data, it relies on massive visual diversity to force the network to ignore the synthetic nature of the training set. Our work differentiates itself by focusing on targeted physical traits rather than pure randomization. By utilizing an extensible 3D Blender pipeline to render high-fidelity physical phenomena specific to chessboards such as specular highlights on wooden pieces and grain in a wooden chessboard, we demonstrate that realistic, physically grounded rendering is crucial in zero-shot transfer tasks.

2.3 Logically Constrained Neural Networks

Many contemporary approaches attempt to teach neural networks the rules of their environment implicitly through customized loss functions or by embedding differentiable optimization layers directly into the training loop (Amos and Kolter [1]). However, these implicit, gradient-based methods rarely guarantee absolute compliance with rigid structural rules. Our approach differs by strictly decoupling the visual feature extraction from the logical constraints. By formulating the structural rules of a chessboard as an Integer Linear Programming (ILP) problem, we apply logic as mathematical constraints on top of the network's output logits. This guarantees legal board configurations and resolves visual ambiguities without requiring the network to implicitly learn the rules of the game.

3 Methodology

3.1 Synthetic Dataset Generation

To bridge the visual domain gap between simulation and reality, we engineered a parameterized 3D rendering pipeline using the Blender Python API [2]. This pipeline programmatically generates high-fidelity chessboard images from Forsyth-Edwards Notation (FEN) strings. The generation process was designed to accurately model real-world chessboard images by incorporating realistic physical traits and domain randomization to ensure robust feature extraction:

- **Material Generation:** Rather than relying on static and clean image textures that neural networks can easily overfit, we generate the surface textures for both the board and the pieces using realistic textures while randomizing some of the texture generation parameters. We utilize wave and noise functions to dynamically generate realistic wood grain for the chessboard. Chess pieces are assigned dynamic wood materials with varied specular highlights; for example, black pieces are stochastically assigned either highly glossy or matte finishes to mimic different manufacturing styles and simulate unpredictable real-world reflections.

- **Empirical Lighting Calibration:** we empirically tuned the physically-based lighting parameters to closely match the ambient conditions of our real-world target domain. The pipeline employs a static dual-area-light setup - a 5200K key light and a 4200K fill light - to cast geometrically accurate, realistic soft shadows across the board.

- **Perspective Randomization:** To simulate the imperfect nature of physical chess games, active pieces are subjected to stochastic yaw rotations (up to 95 degrees around the Z-axis). Furthermore, the camera position undergoes spatial jittering in translation and rotation. For each board state, multiple views are rendered (overhead, east, and west) calibrated

to the chosen player's perspective (white or black), ensuring the network learns perspective invariance for the board view angle and piece rotations.

- **Sensor Noise Simulation:** To prevent the model from overfitting to the clean, noise-free pixels typical of 3D rendering, we deliberately degrade the output images to simulate physical camera sensors. By disabling the rendering engine's denoising algorithms, we introduce and preserve realistic sensor noise. Additionally, we apply a dithering intensity of 0.6 and force heavy JPEG compression. This introduces realistic compression artifacts and edge degradation, forcing the neural network to extract robust, high-level geometric features rather than relying on perfect synthetic edges.

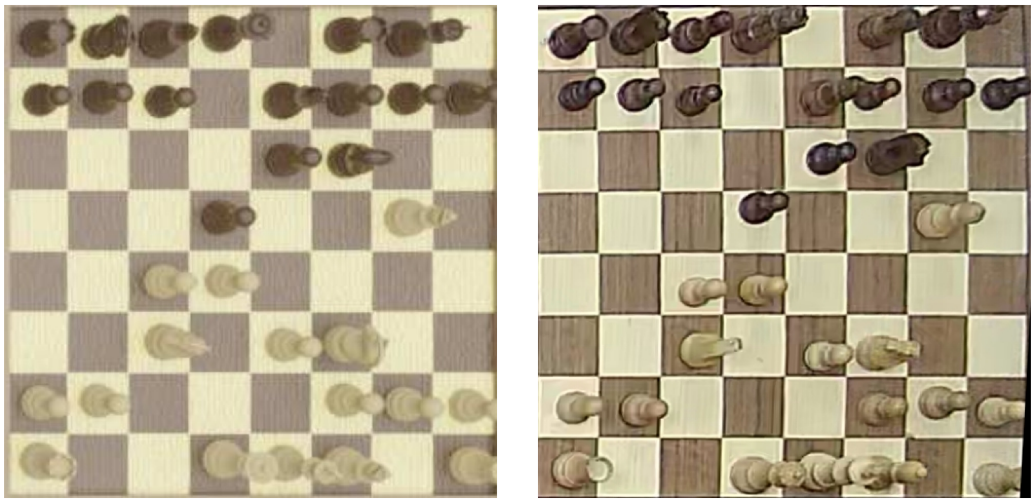

Figure 1: Visual comparison between a synthetic chessboard rendered using our Blender generation pipeline (left) and a real-world chessboard image from our collected dataset (right).

3.2 Input Representation and Preprocessing

Given a full RGB chessboard image, directly predicting the global board state presents a highly complex optimization challenge due to the massive combinatorial output space. To reduce this complexity, we decompose the problem into 64 discrete per-square classification tasks. During preprocessing, the full 480×480 pixels chessboard image is divided into an 8×8 grid to isolate the 64 individual squares. To ensure the network retains critical spatial context such as the overlapping tops of tall pieces (e.g., Kings and Queens) or cast shadows from adjacent squares, we apply a padding transform to each square crop. This allows the localized crops to overlap slightly, capturing visual information that

extends beyond the strict geometric boundaries of a single square, where the amount of padding is determined by a padding hyperparameter whose value is relative to half the base square size. Each padded crop is then resized to our standard input resolution of 112×112 and normalized with pre-calculated dataset mean and standard deviation, resulting in an ordered sequence of 64 image crops, $X = \{x_1, x_2, \ldots, x_{64}\}$, where each $x_i \in \mathbb{R}^{3\times112\times112}$.

Figure 2: Visual comparison of input preprocessing using varying crop padding ratios. From left to right, the square is extracted with fractional padding values of 0.2, 0.8, and 1.4 relative to half the base square size.

3.3 Model Architecture

Our core architecture is a hybrid sequence model designed to capture both fine-grained local textures and global spatial relationships.

- **Local Feature Extraction (ConvNeXt-T)**: To extract robust visual features, we utilize a ConvNeXt-T backbone. Processing each square x_i independently, the ConvNeXt model functions as a high-capacity localized feature extractor. This yields a sequence of localized feature embeddings $F = \{f_1, f_2, \ldots, f_{64}\}$, where each $f_i \in \mathbb{R}^{768}$.
- **Global Context Routing (Transformer Encoder)**: Evaluating pieces in strict isolation ignores global contextual clues. To address this, we treat the sequence of extracted features F as tokens. We inject learned positional embeddings to retain spatial coordinates and pass the sequence through a multi-head self-attention Transformer Encoder. This synthesizes global board context, outputting contextualized embeddings that a linear classification head maps to raw output logits $L \in \mathbb{R}^{64\times13}$ (representing the 12 piece classes plus the empty square class).

3.4 Logically Constrained Inference (ILP Solver)

A standard neural network outputs probabilistic logits that frequently result in illegal chess configurations. To strictly enforce the rules of chess, we formulate

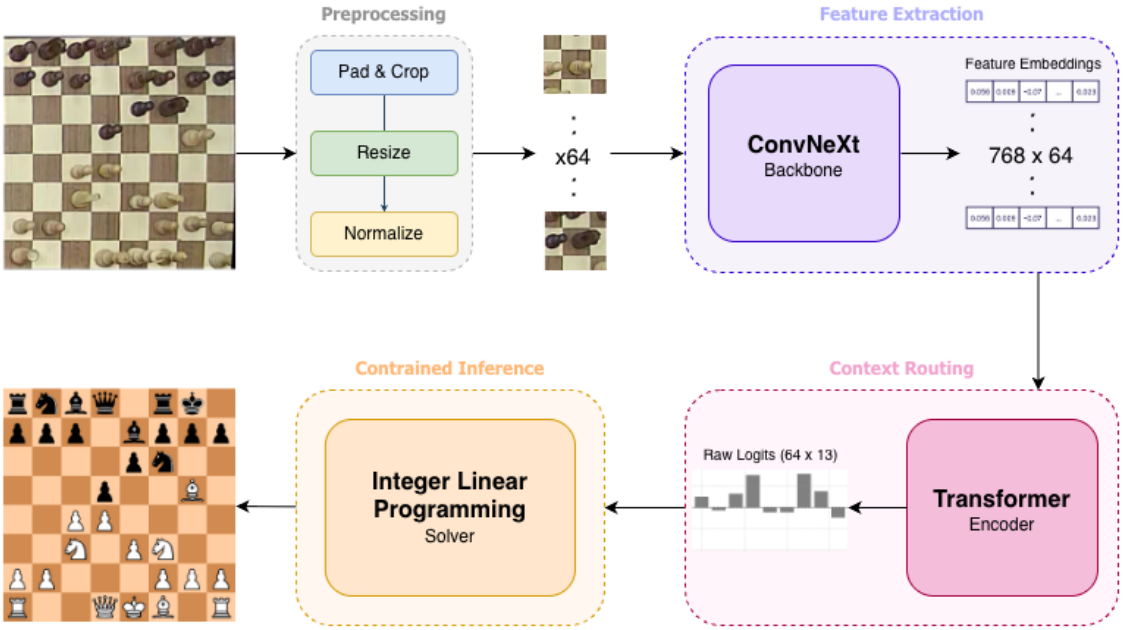

Figure 3: Overview of the proposed Zero-Shot architecture. The pipeline consists of four distinct phases: (1) extracting padded localized square crops from the physical board, (2) generating robust spatial feature embeddings using a ConvNeXt-T backbone, (3) synthesizing global piece relationships via a Transformer Encoder, and (4) applying an Integer Linear Programming solver to the raw logits to guarantee a mathematically valid output board state."

the final decision layer as an Integer Linear Programming (ILP) problem.

Let $x_{i,c} \in \{0, 1\}$ be a binary decision variable indicating whether square $i \in \{0, \ldots, 63\}$ contains piece class $c \in \{0, \ldots, 12\}$. Let $l_{i,c}$ be the corresponding raw logit output from the neural network. The objective is to maximize the sum of the logits for the chosen board configuration:

$$\max \sum_{i=0}^{63} \sum_{c=0}^{12} x_{i,c} l_{i,c}$$

This maximization is subject to the following hard mathematical constraints:

1. **Single Occupancy**: Each square must contain exactly one piece or be explicitly empty.

$$\sum_{c=0}^{12} x_{i,c} = 1 \quad \forall i$$

2. **King Count**: There must be exactly one White King ($c = 5$) and one Black King ($c = 11$).

$$\sum_{i=0}^{63} x_{i,5} = 1, \quad \sum_{i=0}^{63} x_{i,11} = 1$$

3. **Maximum Piece Counts**: The total number of pieces per color cannot exceed a standard chess set (16 pieces per color). Let $W = \{0..5\}$ represent White classes and $B = \{6..11\}$ represent Black classes.

$$\sum_{i=0}^{63} \sum_{c \in W} x_{i,c} \leq 16, \quad \sum_{i=0}^{63} \sum_{c \in B} x_{i,c} \leq 16$$

4. **Specific Class Limits**: While pawn promotion theoretically allows piece counts to exceed standard limits, such configurations are statistically rare. Empirically, strictly bounding the maximum piece counts per color to their initial starting configurations (8 Pawns, 1 Queen, 2 Rooks, 2 Knights, and 2 Bishops) yields a substantial net improvement in overall accuracy by aggressively filtering out false-positive piece predictions.

5. **Invalid Pawn Locations**: Pawns ($c = 0$ and $c = 6$) cannot exist on the 1st or 8th ranks.

$$x_{i,0} = 0, \; x_{i,6} = 0 \quad \forall i \in \{0..7\} \cup \{56..63\}$$

By applying these constraints directly over the network's logits, the ILP solver acts as a logic filter, mathematically guaranteeing a valid final board state.

3.5 Training Procedure and Loss Functions

The training methodology is divided into two distinct phases to effectively close the Sim-To-Real gap:

- **Phase 1 (Zero-Shot Synthetic Training)**: Training on our synthetic dataset was conducted in two sequential steps to ensure stability. First, the ConvNeXt backbone and a linear classification head were trained to convergence using Cross-Entropy loss, acting as a pre-training step for localized feature extraction. Second, the ConvNeXt weights were frozen, and the Transformer Encoder was trained on top of these extracted embeddings to learn global spatial relationships.

- **Phase 2 (Real-World Domain Adaptation)**: When transitioning to real-world images, empirical results indicated that the pure convolutional features transferred much more predictably to noisy real-world textures than the sequence model. Therefore, we discarded the Transformer Encoder for this phase. Resuming from the Phase 1 ConvNeXt pre-trained

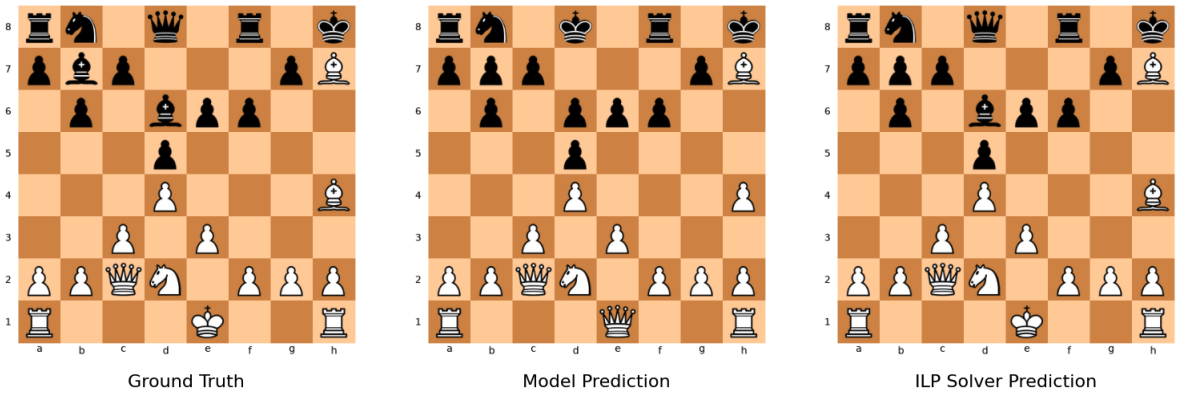

Figure 4: Visual comparison of board state predictions before and after logical constraint filtering. From left to right: The ground truth configuration, the unconstrained raw predictions from our top performance model, and the structurally valid final state produced by the ILP solver. Note how the solver successfully corrects impossible piece configurations present in the raw output - such as 9 white pawns, 9 black pawns, 2 white queens and 2 black kings - to reconstruct the true board state (corrected squares are d8, d6, e1 and h4).

weights, we surgically unfroze only the final convolutional stage of the backbone. We fine-tuned this final stage alongside the pre-trained classification head on our real-world dataset. This localized unfreezing successfully adapted the network to the real-world images without discarding the features learned in simulation.

4 Experiments and Results

4.1 Datasets and Splits

To evaluate our Sim-To-Real pipeline, we utilized three distinct datasets, carefully partitioned to prevent data leakage and evaluate generalization.

- **Naive Synthetic Dataset (Baseline):** Generated using a basic, low-fidelity Blender rendering script. The training set comprises 1,125 images derived from 375 unique FEN strings sourced from three documented chess matches, rendered from three perspectives. The validation set contains 243 images derived from 81 unique FEN strings from a strictly separate fourth match to prevent positional data leakage.
- **High-Fidelity Synthetic Dataset (Phase 1):** Generated using our custom Blender pipeline (detailed in Section 3.1). The training set consists of 1,800 images derived from 600 randomly generated, unique FEN board states across three perspectives. The validation set mirrors the

baseline validation structure, containing 243 images from 81 FEN strings of a documented match.

- **Real-World Target Dataset (Phase 2):** To conduct fine-tuning and testing, we curated a dataset of real-world chessboard images. The training set consists of 238 images sourced from three documented games, each captured from a distinct perspective. Due to the scarcity of annotated real-world games, the validation set (67 images) was drawn from the same three matches but strictly partitioned temporally: frames recorded prior to a specific timestamp were assigned to the training set, while subsequent frames were reserved for validation. Finally, the holdout test set comprises 204 images sourced from three completely distinct matches not seen in training or validation, ensuring a rigorous evaluation of unseen domain transfer.

4.2 Evaluation Metrics

Given the structured nature of the chessboard, we evaluate model performance using two distinct metrics:

- **Per-Square Accuracy (Primary):** The ratio of individual 8×8 bounding boxes correctly classified into their ground-truth label (12 piece classes or the empty square). This serves as the primary metric for our evaluation.
- **Board-Level Accuracy (Secondary):** A strictly binary metric indicating whether the entire board state (all 64 squares) was predicted perfectly. Due to the combinatorial nature of the board space, a single misclassified square out of 64 results in a score of zero for the entire image. This metric directly aligns with the ultimate functional objective of the task: generating an accurate and usable board state for game analysis. Because a single misclassified square can render an entire chess position useless, this metric highlights the difficulty of the end goal. Moreover, this metric serves as a crucial indicator of global structural understanding and the effectiveness of the ILP solver.

4.3 Implementation Details

All models were implemented using the PyTorch deep learning framework and trained on a single NVIDIA GeForce GTX 1080 Ti GPU. Across all primary experimental phases, network weights were updated using the Adam optimizer. To balance GPU memory constraints with gradient stability, training was standardized to a board-level batch size of 2, equating to 128 localized per-square crops processed during each forward pass. While the foundational hardware and optimization framework remained constant, specific training hyperparameters - including initial learning rates, epoch counts, crop padding values, and the application of learning rate schedulers - were empirically tuned for each

distinct experimental phase. Consequently, these phase-specific hyperparameter configurations are detailed within their respective experimental subsections below.

4.4 Experimental results

To systematically evaluate our pipeline, we report zero-shot transfer and fine-tuning performance on our holdout real-world test set. We evaluate the progression of our methodology across three distinct stages: establishing the domain gap, zero-shot generalization, and real-world domain adaptation.

4.4.1 Establishing the Domain Gap

To quantify the Sim2Real domain gap, we established a baseline using a standard ResNet18 trained exclusively on the Naive Synthetic dataset. This baseline was trained for 8 epochs with no padding (i.e. 0.0 padding) and an initial learning rate of 0.0001 to convergence. When evaluated zero-shot on the real-world test set, this baseline achieved a per-square accuracy of just 63.42%. To test the hypothesis that the synthetic data lacked sufficient quality degradation, we applied a suite of data augmentations (Gaussian blur, random noise, and color jitter) to the naive training images. This resulted in an increased accuracy of 66.89%. While this improvement confirmed that a lack of real-world physical textures, noise, and low image quality was a primary failure point, the low overall accuracy demonstrated that rudimentary image augmentations are insufficient to bridge the domain gap, motivating our custom rendering pipeline. We also extracted and compared intermediate network feature maps for both domains. While the feature maps generated from the synthetic data were highly structured and clearly delineated piece structures, the corresponding activation maps for the real-world images were chaotic, distorted, and heavily degraded, confirming that the network was failing to extract coherent geometric features from physical textures.

4.4.2 Zero-Shot Generalization

Replacing the naive data with our high-fidelity synthetic dataset yielded immediate improvements. Training the exact same ResNet18 baseline on the high-fidelity data resulted in 72.10% per-square accuracy on the real-world test data. From this stable data foundation, we empirically optimized the crop padding hyperparameter by training the baseline ResNet18 with multiple fractional padding values (0.0, 0.2, 0.4, ... , 1.4) and evaluating on the real-world test data while using a fixed seed value to eliminate effects of randomness in the training process. Larger padding values allow localized square crops to capture overlapping adjacent piece structures. This experiment pushed the baseline accuracy to 76.64% when choosing a padding value of 1.0, which we used in all subsequent experiments. Applying our ILP solver to these logits further rectified

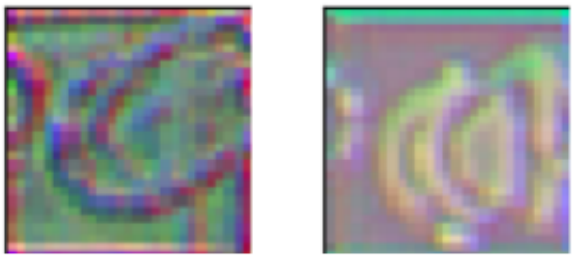

Figure 5: Visualization of intermediate feature maps from the baseline ResNet18 model, illustrating the Sim-To-Real domain gap. The layer activations extracted from a real-world physical chessboard input image (left) exhibit severe noise and distortions. In contrast, the activations from a naive synthetic rendering (right) maintain clean, coherent geometric structures, highlighting the network's inability to generalize to the real-world domain.

errors, achieving 80.92% accuracy. Having optimized the data and preprocessing, we upgraded the backbone model to ConvNeXt, trained over 5 epochs using a learning rate of 0.0001 and a Cosine Annealing learning rate scheduler, which yielded a massive leap to 86.86% zero-shot per-square accuracy. Finally, we trained the Transformer Encoder for 2 additional epochs to convergence, to establish global spatial routing across the ConvNeXt features, which pushed our final Phase 1 zero-shot performance to a final 87.55%. All of the above results are summarized in table 1.

Table 1: Zero-shot transfer accuracy evaluated across architectures and training configurations.

Architecture	**Training Data**	**Augmentation**	**Padding**	**Accuracy(%)**
ResNet18	Naive	None	0.0	63.42
ResNet18	Naive	Blur, Noise, Jitter	0.0	66.89
ResNet18	High-Fidelity	None	0.0	72.10
ResNet18	High-Fidelity	None	1.0	80.92
ConvNeXt-T	High-Fidelity	None	1.0	86.86
ConvNeXt-T + Transformer	High-Fidelity	None	1.0	87.55

4.4.3 Real-World Domain Adaptation

While 87.55% represents exceptional zero-shot performance in our opinion, achieving production-level reliability requires adaptation to the real-wold domain. Using our limited real-world training set (238 images), we evaluated targeted fine-tuning strategies on the ConvNeXt backbone. First, freezing the entire pre-trained ConvNeXt backbone and fine-tuning strictly the linear classification head for 15 epochs with a learning rate of 0.001 and a Cosine Annealing learning rate scheduler yielded a remarkable 96.65% per-square accuracy, proving that the features learned from the synthetic data were already highly robust. Second, surgically unfreezing the final convolutional stage of the ConvNeXt backbone and fine tuning it alongside the classification head using the same hyperparameters and learning rate scheduler allowed the network to adapt even more, achieving a final per-square accuracy of 97.27% and a full-board accuracy of 25.5%. Finally, we conducted a joint-training experiment. Rather than fine-tuning a frozen backbone, we initialized the ConvNeXt model with standard ImageNet weights and trained the entire architecture end-to-end on mixed batches of both synthetic and real-world training images. To facilitate this, we implemented a joint data-loading strategy that dynamically fuses the datasets at the batch level. During each training step, batches from the synthetic data and the real-world data are concatenated along the batch dimension. Because the synthetic dataset is significantly larger, a training epoch is defined by a single complete pass over the synthetic data, while the limited real-world dataset is continuously cycled to guarantee a mixed domain distribution in every forward pass. We chose a batch size of 4 synthetic chessboard images and 2 real-world chessboard images. This joint-training approach yielded a notable empirical trade-off: while per-square accuracy dropped slightly to 95.94%, the board-level accuracy surged to 32.35%. Although not analyzed, we hypothesize that during joint training, the network slightly overfits to the visual characteristics of the real-world training images. Consequently, when evaluating test images that share similar physical configurations and camera angles, the network predicts the entire board flawlessly. Errors therefore become highly concentrated on less familiar images, sacrificing a small degree of per-square accuracy but yielding a higher volume of perfectly classified boards. All of the above results are summarized in table 2.

Table 2: Comparison of real-world domain adaptation strategies. We compare strict linear probing against targeted unfreezing of the backbone's final stage and full end-to-end joint training.

Strategy	**Backbone State**	**Square Accuracy(%)**	**Board Accuracy(%)**
Linear Probing	Frozen	96.65	16.67
Stage-4 Unfreezing	Final Stage Fine-Tuned	97.27	25.49
Joint Training	End-To-End Trained	95.94	32.35

4.5 Ablation Study

To empirically validate our architectural decisions, we isolated specific modules within our pipeline to quantify their independent contributions to overall system performance.

4.5.1 Transformer Encoder

To evaluate the necessity of global board context, we isolated the impact of the Transformer Encoder during Phase 1 zero-shot evaluation. Relying purely on the ConvNeXt backbone as an isolated local feature extractor yielded a raw per-square accuracy of 84.78%. Appending the Transformer Encoder to route self-attention across the sequence of 64 localized embeddings improved this raw accuracy to 85.81%. (Applying the ILP solver to these networks boosted final accuracies to 86.86% and 87.55%, respectively). This improvement supports our hypothesis: while deep convolutional networks excel at localized texture recognition, they lack the receptive field to independently resolve highly ambiguous squares. The self-attention mechanism successfully synthesizes global board context, allowing the network to leverage adjacent piece structures to resolve visual ambiguities prior to any mathematical logic filtering.

4.5.2 ILP Solver

To quantify the impact of embedding strict mathematical rules into the inference pipeline, we evaluated predictive performance across multiple architectures with and without the Integer Linear Programming (ILP) solver. Because standard neural networks evaluate square classifications probabilistically and often without any global board context, they frequently output illegal board states (e.g. multiple white kings). By strictly enforcing the structural constraints of chess (detailed in Section 3.4), the ILP solver functions as a post-extraction logic filter.

Table 3: Performance gains achieved by integrating the Integer Linear Programming (ILP) solver across various zero-shot and fine-tuned architectures.

Architecture	**Phase**	**Raw Accuracy(%)**	**Solver Accuracy(%)**	**Gain(%)**
ResNet18	Zero-Shot	76.64	80.92	4.28
ConvNeXt	Zero-Shot	84.78	86.86	2.08
ConvNeXt + Transformer	Zero-Shot	85.81	87.55	1.74
ConvNeXt (Frozen)	Fine-Tuning	94.69	96.65	1.96
ConvNeXt (Unfrozen)	Fine-Tuning	95.87	97.27	1.40

As demonstrated in Table 3, applying the solver over the network's raw output logits consistently yielded significant accuracy improvements across all tested

architectures without requiring any additional model training. Notably, while the absolute numerical gain provided by the ILP solver naturally decreases as the base visual architecture becomes more robust (e.g., yielding a +4.28% gain for the zero-shot ResNet18 versus a +1.40% gain for our ultimate fine-tuned ConvNeXt), its functional importance remains paramount. As the visual feature extractor adapts to the target domain, it makes fewer obvious categorical errors, leaving only the most highly ambiguous, edge-case visual errors for the solver to mathematically rectify. Ultimately, the ILP solver was required to push our final architecture to its peak performance of 97.27%.

5 Unsuccessful Approaches

To further justify our architectural framework, we document a notable unsuccessful approach regarding the visual backbone. Given the state-of-the-art representation learning capabilities of self-supervised Vision Transformers, we hypothesized that replacing the ConvNeXt backbone with DINO-pretrained ViT models (specifically ViT-Small and ViT-Base) would extract richer, more robust features from our synthetic data. We evaluated these architectures in both isolated classification and hybrid Transformer Encoder configurations. However, across all trials, the DINO backbones yielded inferior performance compared to ConvNeXt. We attribute this to differences in inductive biases: while convolutional networks possess inherent translation invariance that highly optimizes the detection of localized geometric piece structures, Vision Transformers typically require substantially larger data regimes to deduce these spatial priors. Consequently, within the constrained volume of our synthetic dataset (1,800 images), the ConvNeXt architecture proved significantly more efficient and robust for this specific domain transfer task.

6 Limitations

In this work, the size of the synthetic dataset and the complexity of the rendered physical phenomena were explicitly bounded by available computational resources and time limitations. Nevertheless, our Blender rendering pipeline is designed to be highly extensible, and we hypothesize that our zero-shot transfer accuracy could be significantly elevated by expanding the pipeline. Future work should focus on scaling the dataset volume, incorporating broader randomization bounds and engineering additional physical characteristics - such as incorporating a wider variety of materials, diverse piece geometries, and additional lighting conditions.We anticipate that pushing the simulation quality and synthetic data diversity further would close the visual domain gap even more effectively, potentially enabling production-ready zero-shot deployment without the need for any physical fine-tuning.

7 Conclusion

In this work, we presented a comprehensive pipeline for chessboard state recognition that successfully overcomes the simulation-to-reality domain gap. Our methodology demonstrates that robust real-world transcription can be achieved using an extremely limited physical dataset through three core contributions: a parameterized 3D synthetic data generation pipeline, a hybrid ConvNeXt and Transformer architecture for localized and global feature extraction, and an Integer Linear Programming (ILP) solver that functions as a post-extraction logic filter. By systematically addressing the Sim-To-Real gap and mathematically guaranteeing structural validity, our framework successfully resolves the visual ambiguities inherent to physical environments.

References

[1] Brandon Amos and J. Zico Kolter. "OptNet: Differentiable Optimization as a Layer in Neural Networks". In: *CoRR* abs/1703.00443 (2017). arXiv: 1703.00443. URL: http://arxiv.org/abs/1703.00443.

[2] *Blender Python API*. URL: https://docs.blender.org/api/current/index.html.

[3] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *CoRR* abs/1512.03385 (2015). arXiv: 1512.03385. URL: http://arxiv.org/abs/1512.03385.

[4] Zhuang Liu et al. "A ConvNet for the 2020s". In: *CoRR* abs/2201.03545 (2022). arXiv: 2201.03545. URL: https://arxiv.org/abs/2201.03545.

[5] Athanasios Masouris and Jan van Gemert. "End-to-End Chess Recognition". In: *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. SCITEPRESS - Science and Technology Publications, 2024, pp. 393–403. DOI: 10.5220/0012370200003660. URL: http://dx.doi.org/10.5220/0012370200003660.

[6] Joshua Tobin et al. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". In: *CoRR* abs/1703.06907 (2017). arXiv: 1703.06907. URL: http://arxiv.org/abs/1703.06907.

[7] Ashish Vaswani et al. "Attention Is All You Need". In: *CoRR* abs/1706.03762 (2017). arXiv: 1706.03762. URL: http://arxiv.org/abs/1706.03762.

[8] Wikipedia contributors. *Forsyth–Edwards Notation — Wikipedia, The Free Encyclopedia*. 2026. URL: https://en.wikipedia.org/w/index.php?title=Forsyth%E2%80%93Edwards_Notation&oldid=1337978110.

[9] Georg Wölflein and Ognjen Arandjelovic. "Determining Chess Game State From an Image". In: *CoRR* abs/2104.14963 (2021). arXiv: 2104.14963. URL: https://arxiv.org/abs/2104.14963.